2-3 · MongoDB 구축 및 비정형 로그 적재 · 실습 1H

Docker로 MongoDB 기동과 mongosh 기본 조작

컨테이너를 띄우고 넣고·찾고·고치고·지우는 한 바퀴

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

실습 목표

1시간 뒤에는

① docker run 으로 MongoDB 7.0 컨테이너 de-mongo 를 띄운다

② mongosh 로 접속해 logdb 를 만들고 컬렉션에 문서를 넣는다

③ find 조건·점 표기법·$set·deleteOne 을 한 바퀴 돌린다

④ 필드 이름을 일부러 틀려 넣어 DB 가 막지 않는 것을 본다

필드 구성이 서로 다른 문서들이 같은 컬렉션에 들어간다는 점이 오늘의 핵심입니다

흐름

여섯 단계 — 기동부터 일부러 틀리기까지

코드 전문은 학습 노트에 있습니다 — 화면에는 핵심 줄과 확인 포인트만

1. 기동	2. 접속	3. 넣기	4. 찾기	5·6. 고치고 틀리기
docker run de-mongo	mongosh use logdb	insertMany 네 문서	find 조건 점 표기법	$set·deleteOne Level 오타

1·2단계

기동과 접속 — 핵심 명령

1 **docker run -d --name de-mongo ... mongo:7.0**
-p 27017:27017 · -v de-mongo-data:/data/db — 볼륨을 붙여 데이터를 남긴다

2 **docker exec de-mongo mongosh --quiet --eval "...ping..."**
1 이면 준비 완료 — 켜자마자는 ECONNREFUSED, 1초 뒤 다시(실측)

3 **docker exec -it de-mongo mongosh**
첫 화면 경고 세 줄은 정상 — test> 에서 db.version() 은 7.0.43

4 **use logdb**
없으면 만들어 준다 — 첫 문서를 넣기 전엔 show dbs 에 안 보이는 것이 정상

계정·비밀번호

기본은 없음

운영은 --auth

3단계

insertMany — 필드 구성이 다른 네 문서를 한 컬렉션에

path · user	level	status	이 문서에만 있는 필드
/ · u001	INFO	200	—
/login · u002	INFO	200	—
/api/pay · u002	ERROR	500	error: { code: 'PAY-42', message: 'timeout' }
/search · u003	WARN	200	elapsed_ms: 3200

countDocuments() 가 4, findOne() 에 자동으로 붙은 _id 가 보이면 성공 — error·elapsed_ms 유무가 서로 다른데 같은 컬렉션

4단계

find — 조건 문법 한눈에

하려는 일	쓰는 것	결과
조건 일치	find({ level: 'ERROR' })	1건
중첩 필드	find({ 'error.code': 'PAY-42' })	점 표기법 · 1건
비교 연산자	{ status: { $gte: 400 } }	$gt $gte $lt $lte $ne · 1건
목록 포함	{ level: { $in: ['WARN', 'ERROR'] } }	WARN·ERROR
필드 존재	{ elapsed_ms: { $exists: true } }	WARN 1건
필드 고르기	find({}, { _id: 0, level: 1, path: 1 })	1=보임 · 0=숨김
정렬·제한	.sort({ ts: -1 }).limit(2)	-1 = 내림차순

점 표기법으로 중첩 필드를 바로 조회할 수 있다는 것이 표와 크게 다른 점

5단계

고치고 지우기 — $set 을 습관으로

1. **$set — 필드 추가·변경**
   updateOne({ user: 'u003' }, { $set: { reviewed: true } })

2. **$inc — 숫자 증가**
   elapsed_ms 3200 → 3300 · updateMany 로 여러 건, $unset 으로 필드 제거

3. **deleteOne — 먼저 find 로 대상 확인**
   u001 을 지우면 countDocuments() 가 3

4. **$set 을 빼면**
   MongoInvalidArgumentError: Update document requires atomic operators 로 막힌다(실측)

> **확인: reviewed: true 가 붙고 elapsed_ms 가 3300, 마지막 countDocuments() 가 3 이면 성공**

6단계 · 일부러 틀리기

Level 로 넣고 level 로 찾는다 — 스키마가 없다는 것의 실체

넣을 때

insertOne({ Level: 'ERROR', path: '/oops' })

- 오류 없이 그냥 들어간다
- DB 는 필드 이름을 검사하지 않는다

VS

찾을 때

countDocuments({ level: 'ERROR' })

- 2 가 아니라 1 — 오타 문서는 오류도 없이 빠진다
- find({ path: '/oops' }) 로 실제 필드 이름을 확인하는 습관
- 정리: deleteOne({ path: '/oops' })

확인 포인트

여기까지 됐으면 성공

1. docker ps 에 de-mongo 가 Up · ping 이 1
2. db.version() 이 7.0.43 · use logdb 뒤 db 가 logdb
3. countDocuments() 가 4 → 삭제 뒤 3, findOne() 에 _id 자동
4. ERROR 1건 · $gte 400 1건 · $exists 는 WARN 1건
5. Level 오타 문서가 들어가고 level 로 세면 1 — 오류 없이 빠진다

적재 코드에서 필드 이름을 상수로 관리해야 하는 이유가 여기 있습니다

2-3 · 실습 1H 마무리

다음은 로그 적재와 집계 실습 (1.5H)

파이썬(pymongo)으로 1,000건을 넣고 aggregate 로 집계한 뒤, 인덱스 전후를 재 봅니다

- ✓ 가상환경 .venv 와 pip install pymongo
- ✓ load_logs.py 로 1,000건 insert_many
- ✓ $group·$sort·$dateToString 집계
- ✓ 인덱스 전후 totalDocsExamined 비교

준비할 것

mongosh 창은 두고 새 터미널 하나 더